0911

三个消融批次合并成一个套件自动跑

前两组(homefilter、timeratio)的可视化检查通过后,把第三组也加了进来,三个批次用一个固定脚本按顺序自动跑完:检查数据是否齐全(缺了才重新采集)→建索引→出可视化→训练→评测→自动写结果表。每批跑完两卡再跑下一批,预计全部结束需要 4-6 天。

第三组叫 combo,把前两组的删除规则叠加:既删起点 2cm 内的配对,又删时间跨度超过平均轨迹长度 20% 的配对。想回答的问题是:空载等待的样本和进度差太远的样本,一起删掉是不是比单独删更好。这里是控制变量设计:两个过滤的参数与单变量组完全一致(0.02m 同 homefilter,0.2 同 timeratio),本组唯一的新变量是「两者同时生效」;最初把时间比例写成 0.15,如果那样跑,结果变好也说不清是叠加的功劳还是参数更紧的功劳,已改回 0.2。另外 timeratio 本身也有过一次改动:比例最初设 60%,后收紧到 20% 并重建了索引,现在跑的是 20% 版本。

删掉的比例在各任务上差别很大,这一点值得留意:0.2 比例下单罐放篮子删了 81%(它恰好是上一批 EMIX 提升最大的任务),叠方块删了约 65%,而有的任务只删 1-2%。如果这批结果变化集中在删除比例大的任务上,就说明时间跨度确实抓到了东西;如果到处都差不多,这个维度可能没那么重要。

目录规范上走了一段弯路:输出一度放到顶层 `runs/` 目录,与"实验输出必须在 outputs/ 下"的既有规范冲突,已改回 `outputs/run_<批次名>/{vis,train,eval,summary}/`,路径校验也恢复为只认 outputs/。已渲染的可视化随之迁移,无损失。

0911

为什么做 homefilter 和 timeratio 这两组实验

现在有一个猜想：**EMIX 可能比较擅长一次抓放，但任务要连续抓放几次时，它可能分不清“已经做完什么、接下来该做什么”，把前面得到的好处丢掉了。** 这个猜想来自 top500-emixfix 这批结果（commit `3d3c51e`，run `flow-20260907-222045`），还没有得到证实。

比如，任务刚开始时，手臂在起点；放完第一块面包后，手臂也可能回到差不多的位置。但这两张图里的面包数量、位置和任务进度不同。把它们叠起来训练，可能会让模型搞不清下一步。即使第一轮抓得更好了，第二轮出错，整个任务还是失败。

因此做下面两组实验。两组都使用原来 top500-emixfix 的数据，只重新训练 EMIX，普通 DP（baseline）沿用归档结果。每组只改一种配对筛选规则，方便看出是哪种改动有用。

第一组：homefilter，去掉公共起点附近的配对。

这组想检查：问题是否主要出在“手臂位置差不多，但任务进度不同”的起点附近。具体做法是删掉距离公共起点小于 2cm 的配对。之前观察到，很多配对集中在手臂空载、返回或等待的位置，真正抓取和放置的位置反而少。

批次名是 `top500-emix-homefilter`。数据从归档复制回 `data/lerobot`，不重新采集。在已检查的三个 500 条轨迹任务里，每条轨迹的起点都一样，约为 `(-0.001,-0.395,0.953)`，标准差为 0；起点 2cm 内的帧占全部帧的 2–6%。所以先把过滤半径设为 `0.02m`，与 `xyz_epsilon` 相同，也可以用 `--exclude_start_radius` 修改。

已实现 `exclude_start_radius`，包括配置、命令行和两种索引构建方式；索引记录中会保存 `start_point_xyz` 和 `start_filter_removed`。计划评测 100 个场景，seed 为 0。与旧 baseline 比较时，要核对实际场景清单；配置里都写 seed 0，不代表两次最终选中的场景一定相同。

第二组：timeratio，去掉时间相差太远的配对。

这组想检查：问题是否来自把“刚开始做”和“快做完了”的画面混在一起。之前的统计发现，38–59% 的配对中，两点在各自轨迹里的进度相差超过 30%。例如，一条轨迹刚出发的画面，可能与另一条轨迹快结束的画面配在一起，因为手臂都在起点附近。

批次名是 `top500-emix-timeratio`。规则是：两点在各自轨迹里的时间位置之差，不能超过本批所有轨迹平均长度的 60%。参数为 `max_pair_time_ratio=0.6`，可用 `--max_pair_time_ratio` 修改。这组不加起点过滤。homefilter 查的是“配对发生在哪里”，timeratio 查的是“两个画面在任务过程里相隔多远”。

本次记录时已完成：功能提交 `f8e555a`、10 个任务的索引重建（暂存 `/tmp/tr_index`），以及 SUMMARY 的初始内容。可视化排在 homefilter 后面，两组共用 flow 锁，按顺序运行。输出放在 `outputs/intersections_timeratio/`，避免和另一批混在一起。

两组过滤都会改变模型看到哪些样本、各类样本出现多少次。**如果成功率提高，也不能马上认定是“进度判断”被修好了，还要看改善发生在第几轮、什么动作上。**

后面还可以试：两张图本身是否真的有足够大的差别。

现在找到的配对满足“手臂姿态接近、接下来动作不同”，但两张图可能看起来几乎一样。如果图里没有足够的信息解释动作为什么不同，强行拿来训练，未必能帮助模型学会看图。后面可以考虑少用这类配对，或者把它们筛掉。

先把配对按图像差异从小到大分组，看看原图、叠起来的图和对应动作。重点看：变的是物体位置、剩余物体、任务进度，还是只有背景、光照、机械臂外观变了。也要比较 homefilter 和 timeratio 是否已经改变了这些配对的比例。

不是图像差得越大越好。 有时只有一个小物体移动了，但它正好决定下一步；有时整张图亮了很多，却与动作无关。如果看图后觉得这个方向有希望，再单独试图像差异筛选。还要和“随机删掉同样数量的配对”比较，确认好处来自筛选规则，而不只是配对变少了。这是后续想法，目前还没做，也不放进正在进行的两组实验里。

哪些结果支持这个猜想，哪些结果不支持

这里一次完整的“抓起来再放下”算一轮，不把接近、抓取、搬运、释放各算一轮。下面的差值都是 EMIX 减去 baseline，单位为百分点。

抓放轮数	任务及差值	任务数	平均差值（每个任务权重相同）
1 轮	单罐放篮子 +15；面包放平底锅 +5	2	+10
2 轮	双鞋放置 +11；双罐放盒子 +5；汉堡薯条放置 +1；叠碗 0；叠方块 −4；面包放篮子 −8	6	+0.83
3 轮	按大小排方块 0；按颜色排方块 −2	2	−1

三个回退任务都要连续处理多个物体，而且轮数越多，这批任务的平均收益越小。这是支持猜想的线索。但双鞋也要做两轮，却提升了 11 点，说明**多轮任务不一定变差，也不一定只能得到很小的提升。**

下一步要在相同评测场景里，分别看第一轮、第二轮、第三轮。最想确认的是：有没有“第一轮比 baseline 好，第二轮反而差”的情况。第二轮成功率要在第一轮已经成功的样本中算，不能把第一轮就失败的样本都当成第二轮失败。还要记录第一次出错发生在哪里：重复抓已经处理的物体、漏掉剩余物体、做完一轮后停住，还是知道该抓谁但没抓准、没放准。

如果第一轮就明显变差，或者主要是抓放不准，就不支持“模型搞不清任务进度”这个解释。两条轨迹动作不同，也不代表它们一定处于不同阶段，更不能直接说其中一个动作是错的。

目前每组只有 2、6、2 个任务，任务难度、相机视角和放置精度要求也不同。按大小排序的任务，两种方法都只有 3%，几乎都没学会，也很难据此比较。还没有换多个训练随机种子重复实验，所以这些平均数只能提出问题，不能证明原因。

抓放轮数来自归档专家数据里夹爪开闭的统计，成功率来自原始 metrics。详细数据见当前实验检查和历史结果对比。

0911

500 条轨迹、俯视相机这批实验已经归档

批次名为 `top500-emixfix`。归档位置是 `archive/DP-10tasks-500ep-emix-baseline-cen-arm-top-cam-20260911`，commit 为 `3d3c51e`，run 为 `flow-20260907-222045`。

baseline 总成功率为 43.8%，EMIX 为 46.1%，相差 **+2.3 个百分点**。之前 200 条轨迹那批是 −2.9 点，这次变成正数。说明扩大工作区后 EMIX 并不是在所有情况下都没用，但还不知道这次改善分别有多少来自数据增加、配对规则变化和索引构建修复。

按实际差值统计是 5 个任务提高、3 个降低、2 个持平。旧记录写成了“5 胜 2 负 3 平”，这里更正。单罐放篮子 +15、双鞋 +11 值得关注；面包放篮子仍低了 8 点，之前 200 条轨迹时低了 11 点。按大小排序双方都是 3%，任务本身仍没有学好。

旧记录用“超过 ±5 点”判断结果是否可信，这个办法没有统计依据。现在只能把这些当作本批观察到的差异，还不能说已经证明稳定提升或回退。和 200 条轨迹相比，baseline 平均提高 2.5 点、EMIX 提高 7.7 点，两者差距改善了 5.2 点；但同时改了多个条件，不能把它全部算到某一个改动上。旧构建在 500 条轨迹规模下无法完成，所以也没有“500 条轨迹＋旧构建”的结果可比较。

每个任务评测 100 次，配置 seed 为 0。数值为成功率百分比，括号里是成功次数：

任务	baseline	EMIX	相差多少个百分点
blocks_ranking_rgb	28 (28/100)	26 (26/100)	−2
blocks_ranking_size	3 (3/100)	3 (3/100)	0
place_bread_basket	45 (45/100)	37 (37/100)	−8
place_bread_skillet	87 (87/100)	92 (92/100)	+5
place_burger_fries	36 (36/100)	37 (37/100)	+1
place_can_basket	49 (49/100)	64 (64/100)	+15
place_cans_plasticbox	20 (20/100)	25 (25/100)	+5
place_dual_shoes	33 (33/100)	44 (44/100)	+11
stack_blocks_two	49 (49/100)	45 (45/100)	−4
stack_bowls_two	88 (88/100)	88 (88/100)	0
全部任务	**43.8** (438/1000)	**46.1** (461/1000)	**+2.3**

归档时已经清空 `data/{robotwin,lerobot,emix_index}` 和 `outputs/{train,eval,intersections}`。模型检查点已删除，释放约 351G；每个评测 run 只保留前 10 段视频。

0907

启动 500 条轨迹的俯视相机实验

这次用修复后的索引构建方式重新跑完整流程。新方式分批处理候选配对，避免一次占用太多内存；每个任务最多保留“轨迹数 × 1000”个候选对，所以 500 条轨迹最多保留 50 万对。这是整个任务的总上限，不是每条轨迹都分到 1000 对。上限和随机种子会写进索引记录。

本批参数为：位置差小于 `0.02m`、姿态差小于 `0.26rad`，并去掉每条轨迹最后 `0.1s` 的配对候选。旧记录说“与 200 条轨迹那批的配对阈值不变”，后来的历史核对发现不对：旧索引实际使用 `0.01rad`。因此两批不只是数据量和构建方式不同。

RGB 和 size 排序直接复用 0813–0906 已有的 500 条轨迹和修复后的索引，每个任务 50 万组，已检查可视化并通过新旧构建结果的一致性测试。其余 8 个任务由 `assets.py` 排队重新采集，每个任务 500 条，当时预计需要 2–3 天。

采集之后由 `run.py` 继续：建立索引（当时预计每任务约 8 分钟）、生成可视化（`outputs/intersections`，HTML 和 PNG）、训练和评测。普通 DP 与 EMIX 各训练 30,000 步，各评测 100 次，seed 为 0。

启动时的记录：

- 主仓库 commit `3d3c51e`，子模块 commit `61798c9`，`episode_num=500`。
- tmux 会话为 `top500emixfix`。
- 命令为 `assets.py --variant cen_arm_top_cam --gpus 0 1 --episodes 500 && run.py --config configs/scripts/run_cen_arm_top_cam_500ep.yaml`。
- 采集日志在 tmux 窗口和 `logs/top500emixfix-tmux.log`，通过 pipe-pane 保存。启动时 flow run ID 尚未生成，需等采集结束后查看 `logs/flow-*`；最终 ID 见 0911 归档记录。

- 详细配置在 `run_cen_arm_top_cam_500ep.yaml` 和各任务的 `train_config_snapshot.json`：`steps=30000`、`batch=64`、`lr=1e-4`、`resnet18`、`horizon=16`、`n_obs=1`、`n_act=8`、`seed=0`。
- EMIX 使用 `alpha=beta(2)`，本体状态取第一个配对样本（anchor），损失权重 `1.0`；选择 EMIX 批次的概率按 logarithmic scheduler 从 `0.6` 降到 `0`。评测保存视频。
- 旧对照归档为 `archive/DP-10tasks-200ep-emix-baseline-cen-arm-top-cam-20260904`，当时 EMIX 平均低 2.9 个百分点。

0904

检查为什么扩大工作区后，EMIX 的好处变少了

当时检查了 right_wide 和 top 两批的配对索引，以及机械臂末端的位姿，发现三个值得追查的问题。下面保留当时的数字，但它们只是线索，还没有证明回退原因。

1. **手腕方向要求可能太严。** 两点位置要相差不到 2cm，手腕方向还要相差不到 `0.01rad`，约 0.57°。当时统计的位置候选通过率有 94–99%，但加上方向要求后只剩 17–66%。保留下来的方向差有 95% 不超过 `0.0099rad`，已经很接近门限。扩大工作区后，手臂从不同方向接近物体，方向更难相同，可能因此更容易选中起点附近，而不是实际操作的位置。
2. **配对可能把不同进度混在一起。** 38–59% 的配对中，两点在各自轨迹里的进度相差超过 30%。例如一条轨迹刚开始，另一条快结束，但手臂碰巧都在起点附近。叠图后同时监督“出发抓东西”和“回去等待”，可能影响模型判断。旧记录直接写成“教出了不动的模式”，现在还没有足够证据确认这一点。
3. **真正抓放的位置选得少。** 当时配对触及的原始帧只有 2–16%，而且集中在没有拿物体的时段。训练初期约 60% 的批次使用 EMIX，这可能让模型花很多时间学这些位置，实际抓取和放置反而练得不够。这里的 60% 是初期概率，不是整个训练的平均比例。

当时的背景是：旧批中面包放平底锅达到 32→95，单罐放篮子达到 44→66；换成居中手臂、扩大工作区和 200 条轨迹后，三个机位批次的平均差值分别为 +1、+0.6、−2.9 点。旧批被记录为约 500 条轨迹、较窄的物体分布，但归档缺少对应数据，不能独立核实所有条件。

旧记录认为“配对看的是机械臂末端位置，所以相机没有影响”，这句话不成立。配对选择可以不依赖相机，但混合出来的图像仍然受视角影响。另外，也不能直接把 ±5 点内的差值都当作没有效果，仍需重复训练和公平评测。

当天做了两个改动，不需要重新采集数据：

- 把 `rpy_epsilon` 从 `0.01` 放宽到 `0.26`，约从 0.57° 放宽到 15°，代码在 `embodied_mixup/configs/config.py`。当时测得方向筛选的通过率：bread_skillet 从 33.4% 到 96.1%，can_basket 从 17.1% 到 72.0%，stack_bowls 从 66.3% 到 98.2%。通过率提高不代表这些配对一定更有用。
- 新增 `exclude_tail_seconds`，默认 `0.1s`，按当时 `dataset_fps=17` 约为末尾 2 帧，让这些帧不参与配对。改动覆盖 config、`candidates/pairs.py` 两种构建方式和 CLI 的 `--exclude_tail_seconds`、`--dataset_fps`；索引的 `config_snapshot` 也记录新参数。这个范围很小，不能保证去掉所有开始与结束的配对；当时考虑必要时再试 1–2 秒。

位置门限仍是 2cm。改动的想法是让实际抓放位置更容易配上对，同时少混入收尾画面。两个参数都可以从命令行修改，旧索引需要重建。当天计划先用归档的 200 条轨迹重建索引，比较配对数量、涉及哪些帧、跨进度配对的比例，再决定是否重训。

0820

修复图像问题后，成功率上升，但部分任务仍然回退

修复 OIDN 错误后，成功率明显上升。第一版结果使用 seed 0，每个任务评测 100 次。以下均为 baseline→EMIX：

- 提高：place_bread_skillet 32→95，place_can_basket 44→66，stack_blocks_two 65→70。
- 降低：blocks_ranking_rgb 38→27，place_bread_basket 70→66，place_burger_fries 73→71。
- 持平：blocks_ranking_size 4→4。
- 写这条记录时，place_cans_plasticbox 和 stack_bowls_two 的 EMIX 还没评完，flow 正在 5060 上跑 stack_bowls_two。

准备对三个回退任务补测 seed 1 和 seed 2，各 100 次，每个任务共补 200 次。这里只换评测随机种子，不重新训练，而且只评 EMIX。baseline 沿用旧结果只能作为参考，不能当成相同场景下的新对照。

新增脚本 `scripts/eval_reseed.sh`，复用 flow 生成的 `emix_eval.yaml`，从命令行覆盖 episodes、seed 和 output-dir。输出放在 `outputs/eval_reseed/<task>_dp_emix_seed{1,2}/`。seed 1 在 4070 的 `emix-twin` 环境立即运行；seed 2 在 5060 的 `emix-twin-5060` 环境排队，等 flow 主进程结束再运行。两个环境都去掉了 `LD_LIBRARY_PATH` 中的 `/usr/local/cuda`。当时说的“卡 0”是 `nvidia-smi 0`，对应 5060，但运行时要用 `CUDA_VISIBLE_DEVICES=1`。

看视频时还发现：EMIX 有时第一次没夹住，会再试一次，而不是一直停住。这可能说明它更能处理执行中的失误，但目前只是视频观察，还没统计重试次数和恢复成功率。

0810

数据多了以后，建立配对索引时内存不够

帧数增加后，需要查找的近邻和候选对太多，导致内存耗尽（OOM）。优化后，单个数据集已经能成功建立 EmixIndex。之后数据更多时，可能还要限制搜索范围，例如只在一部分轨迹里找配对，避免所有数据一起搜索。这还只是后续想法。

此前已完成 10 个任务的单臂宽视角版本。窄视角几乎看不到实际操作区域，所以先换成宽视角；如果重要物体根本不在画面里，要求模型更依赖视觉未必有帮助。HangingMug 当时还没有做好，需要继续调试采集代码。

RoboTwin 默认有 click bell 单臂任务，但 baseline 和 EMIX 都很容易成功，不适合用来区分两种方法，因此继续做其他抓放任务。评估代码默认只接受左臂动作，而新任务可能随机使用左右臂，所以增加任务包装，把相关随机化和操作限制在左侧。

看 200 条轨迹的可视化时，发现能用于配对的交点只有约一百个，因此考虑多采集数据。但改造任务后，专家完成任务的成功率下降，采集 500 条轨迹会很慢。采集失败也可能导致某些位置的数据缺失，让配对更难找到。

增加到 500 条轨迹后，又发现交点大多集中在轨迹末尾。后来调整任务，让场景更多样，但专家成功率也更低，配对情况没有明显改善。怀疑这与任务流程和 RoboTwin 的实现有关，还没有确认。

继续用这批数据训练，baseline 成功率为 88%，EMIX 为 87%。只差 1 点，可能来自随机波动，目前不足以说明两者有稳定差别。